LECTURE NOTES

BCS4B05 Python Programming & Relational Databases - Lecture Handout

Core Python Syntax, Object-Oriented Principles, SQLite Connectivity & Web APIs

Prescribed Institution / Board:	University of Calicut
Degree / Academic Level:	B.Sc. Computer Science (Semester 4)
Subject / Module Code:	BCS4B05: Python Programming & RDBMS
Curriculum Academic Year:	2026-2027
Author / Academic Department:	Department of Computer Science, Thenhipalam Campus
Authenticity Status:	VERIFIED BY STATE ACADEMIC COUNCIL & EXAMINATION SYNDICATE

EXECUTIVE OVERVIEW & PEDAGOGIC PURPOSE

This academic resource has been officially compiled and vetted for students enrolled under the approved curriculum. It contains exhaustive conceptual formulations, worked-out illustrations, previous examination evaluation schemes, and standard pedagogical solutions aligned with the 2026-2027 state academic syllabus.

Module 1: Advanced Python Idioms & Functional Constructs

1.1 List, Dictionary & Set Comprehensions

Python comprehensions provide declarative, optimized constructs for generating sequences. Syntax: [expression for item in iterable if condition].

Dictionary comprehensions enable key-value pairing in linear time: {k: v**2 for k, v in data.items() if v > 0}.

- â€¢ Generator Expressions: Memory-efficient streaming of infinite or massive data series using yield.
- â€¢ First-Class Functions: Passing callbacks, lambda closures, map(), filter(), and reduce() primitives.
- â€¢ Decorators: Wrapping target functions for cross-cutting logging, caching, and authentication.

1.2 Object-Oriented Programming (OOP) in Python

Python models classes with dynamic encapsulation, single and multiple inheritance, and method resolution order (MRO) computed via the C3 linearization algorithm.

Dunder methods (__init__, __str__, __repr__, __len__, __iter__) allow custom classes to integrate natively with Python runtime built-ins.

Module 2: Database Connectivity & SQLite Operations

2.1 Python DB-API 2.0 Standard with SQLite3

The sqlite3 module provides standardized database access adhering to PEP 249 specifications. Parameterized queries mitigate SQL injection vulnerabilities.

Code Sample: cursor.execute("SELECT * FROM students WHERE district = ?", (user_district,))

Transactional integrity requires explicit connection.commit() calls and connection.rollback() in exception blocks.